

Take-home challenge — Web QA automation engineer

You don't need to automate everything. Show judgment, keep it tight, and demonstrate how you think. Aim for about 4–6 hours.

Submission deadline: 3 business days.

All code submitted must be authored by **you** without the use of generative AI. You may use documentation sites, manuals, and StackOverflow, but do not use AI-generated code.

Objective

Demonstrate your ability to design, implement, and execute automated tests for a web application. We're evaluating your test design, framework skills, Cypress/JavaScript proficiency, and how clearly you communicate results.

Application under test

- **URL:** <https://demoqa.com/>
- **Scope:** Cover key user interactions across a few representative pages (forms, selections, dialogs). Prioritize meaningful scenarios over volume.

Note: If anything is flaky or unstable, call it out and show how you handled it.

Tasks

1. Test framework setup

- **Framework:** Cypress with JavaScript (TypeScript optional).
- **Structure:** Clear separation of specs, selectors/page objects, fixtures/test data, and utilities.
- **Execution:** A single command to run the main suite.
- **Config:** Base URL and environment basics (e.g., timeouts).
- **Docs:** README with prerequisites, install, run (headed/headless), and how to view results.
- **Optional:** Basic code quality tooling (ESLint/Prettier).

2. Automated test creation

- **Design:** Implement concise, reliable tests that demonstrate both typical flows and a few edge cases.
- **Selectors:** Prefer resilient locators; show reuse via helpers, page objects, or custom commands.
- **Optional:** Data-driven tests, thoughtful flakiness handling, lightweight accessibility checks.

3. Execution and analysis

- **Artifacts:** Include run output, reports, or screenshots of summary results.
- **Report:** Summary with number of tests, pass/fail, known limitations, and any observed flakiness plus your approach.

4. Defect reporting

- **At least one defect:** Actual or hypothetical from your work.
- **Include:** Title, steps to reproduce, expected vs. actual, severity/priority with rationale, and supporting evidence.
- **Format:** A markdown file (e.g., DEFECTS.md) or a clearly labeled section in your report.

5. Recommendations for improvement

- **Practical suggestions:** How you'd integrate into CI/CD, organize suites, maintain test data and structure, tagging/parallelization, and metrics you'd track.
- **Optional:** A simple CI config (e.g., GitHub Actions) that runs the suite on push.

Submission

- **Repository:** Public GitHub repo with all Cypress code and supporting files.
- **README:** Setup and run instructions.
- **Results:** Test artifacts or a note explaining where they're generated.
- **Summary report (max 1 page):** Your approach, key design decisions, trade-offs, and notable insights or challenges.

Evaluation criteria

- **Test design:** Coverage of important scenarios with thoughtful edge cases.

- **Page Object Model:** Well-designed, reusable page models with centralized selectors.
- **Code quality:** Logical structure, readable and DRY tests, robust selectors.
- **Reusability & utilities:** Good use of custom commands, helpers, and shared logic to avoid duplication and support scalability.
- **Framework:** Ease of setup/run and signs of scalability.
- **Analysis & communication:** Clear documentation, solid defect report, and a useful test results summary.
- **Best practices & initiative:** Meaningful assertions, avoidance of fragile waits, and any thoughtful extras.